# Computer Science
# Partial exam rehearsal – Student version

IGE – 1st Year
Class practice

**Suggested duration:** 1 hour 45 minutes, followed by class discussion.

Answer each question and show your reasoning. For multiple-choice questions, choose one answer and justify it. Write your answers on separate sheets. Questions are in English.

**Content.** This rehearsal covers three units. Unit 1 focuses on binary representation, CPU instructions and register states (2 points, 20%). Unit 2 covers Python imports, numeric operations, Boolean conditions, `if-elif-else`, loops and lists (3 points, 30%). Unit 3 covers Arduino programming, error detection, digital inputs and outputs, and an LCD scoreboard (5 points, 50%). These weights are indicative for this academic year and should be used as a guide when preparing for the exam.

# Unit 1 Problems (2.0 points total)

## Q1 - CPU execution

A processor has four registries, labeled R0, R1, R2, and R3 and the following set of operation codes:

1. `0001` Addition
2. `0010` Substraction
3. `1100` Multiplication
4. `1101` Integer Division
5. `1110` Modulo (reminder after Integer division)

The 10-bit instruction is formatted as follows.

`Opcode(4) | Dest(2) | Src1(2) | Src2(2)`

For example, the instruction `1110 00 01 10` means "calculate the reminder after Integer division of the value of registry R1 divided by the value of registry R2 (R1 modulo R2), and store the value in registry R0".

The initial value in the registers are in table 1:

| | |
|---|---|
| R0 | `0b1011` |
| R1 | `0b1000` |
| R2 | `0b0010` |
| R3 | `0b0000` |

Table 1: Initial states

Answer the following questions:

1. Convert to decimal the values in registry (prefixed with `0b`). **(0.5 pts)**

2. Write the instruction to add the values in registries R1 and R2 and store it back in registry R2. This instruction will be instruction 1. **(0.2 pts)**

3. Consider the following instructions:

   - Instruction 2: 0010000001
   - Instruction 3: 1110110100

   Explain what every instruction will do.(0.3 pts)

4. Using the initial state, apply instruction1, instruction2 and instruction3. Show the state of the registers after the execution of every instruction. **(1 pts)**

# Unit 2 – Python (3 points total)

## Python Q1 – Scenario 4 – Requesting a concise import (0.375 pt)

A financial analyst only needs `sqrt()` from the `math` module and wants to call it without the `math.` prefix. Which option meets the requirement?

A. `import math` followed by `math.sqrt(81)`
B. `import math as sqrt` followed by `sqrt(81)`
C. `from math import sqrt` followed by `sqrt(81)`
D. `from sqrt import math` followed by `sqrt(81)`

**Your choice:**
**Which name becomes available?**

## Python Q2 – Scenario 2 – Packing devices (0.375 pt)

A warehouse must pack 137 devices into boxes that hold 12 devices each. The manager needs the number of complete boxes and unpacked devices. Which solution is most appropriate?

A.

```
boxes = 137 / 12
remaining = 137 / 12
```

B.

```
boxes = 137 // 12
remaining = 137 % 12
```

C.

```
boxes = 137 % 12
remaining = 137 // 12
```

D.

```
boxes = int(137 / 12)
remaining = 137 - 12
```

**Your choice and justification:**

## Python Q3 – Scenario 4 – Secure document access (0.375 pt)

A user may open a confidential document when they have permission and their account is not suspended. The following code illustrates how the condition will be used. Replace `<condition>` with one of the options below. Which condition states the requirement most directly?

```
has_permission = True
account_suspended = False

if <condition>:
    print("Access granted")
else:
    print("Access denied")
```

A. `has_permission or not account_suspended`
B. `has_permission and not account_suspended`
C. `not has_permission and account_suspended`
D. `has_permission and account_suspended`

**Your choice and a test that should deny access:**

## Python Q4 – B7. Diagnosing an index error (0.375 pt)

An AI assistant generates the following loop and Python raises `IndexError` on the last iteration:

```
for index in range(len(values) + 1):
    print(values[index])
```

Which explanation is most accurate?

A. Lists cannot be used inside `for` loops.

B. `range()` includes its stop value.

C. The `+ 1` creates an index equal to the list length, which is outside the valid range.

D. The loop must begin at index `1`.

## Python Q5 – A7. Prioritise outcomes (0.75 pt)

A support request should receive exactly one classification:

1. `"Critical"` when the service is unavailable.
2. `"High"` when data may be lost.
3. `"Medium"` when response time exceeds 5 seconds.
4. `"Normal"` otherwise.

Implement the policy with `if-elif-else`. Create a case where several conditions are true and explain why only one branch runs.

```
service_unavailable = False
possible_data_loss = True
response_time = 8.2

# Implement and print the classification here.
```

## Python Q6 – A5. Preserve a running series (0.75 pt)

A new online service begins with 200 registered users (stored in `initial_users`). A list contains the new users gained each week (`new_users_by_week` stores the new users for each week).

Create `total_users_by_week` so that it contains the initial total followed by the total after every week.

```
initial_users = 200
new_users_by_week = [25, 30, 20, 35]
total_users = initial_users
total_users_by_week = [initial_users]

# Update total_users for every week.
# Preserve every updated total in total_users_by_week.

print(total_users_by_week)
```

# Unit 3 Problems (5 points total)

## Q1 — Error Detection & Correction (1.5)

Consider the following code, it contains several syntax/logic errors.

```
const int led1 = 11
const int led2 = 2;
const int btn = 11;

void setup() {
    pinMode(led1, OUTPUT)
    pinMode(led2, OUTPUT);
    pinMode(btn, OUTPUT);
}

void setup() {
if (digitalRead(btn) = HIGH) {
    pattern = !pattern;
    }
    if (pattern = true) {
        digitalWrite(led1, HIGH);
        digitalWrite(led2, LOW);
    } else {
        digitalWrite(led1, HIGH);
        digitalWrite(led2, HIGH)
        delay(300);
        digitalWrite(led1, LOW);
        digitalWrite(led2, LOW);
    }
    delay(300)
}
```

Solve the following items:

1. Find all 9 errors in the code (0.5 pts).

2. For every error, explain why it's an error and how to fix it (0.5 pts).

3. Explain what the corrected code is doing (0.5 pts).

## Q2 - Referee table (1.5)

The following schematic represents your prototype board. The integrated protoboard includes 2 push buttons, labeled as B1, and B2, and two (pull-down resistors). The objective of the prototype is to provide an electronic table to keep track of the score in a football match. Push-button B1 is used to control the score of the local team and B2 is used to control the score of the visiting team (1 push increases the score in 1).

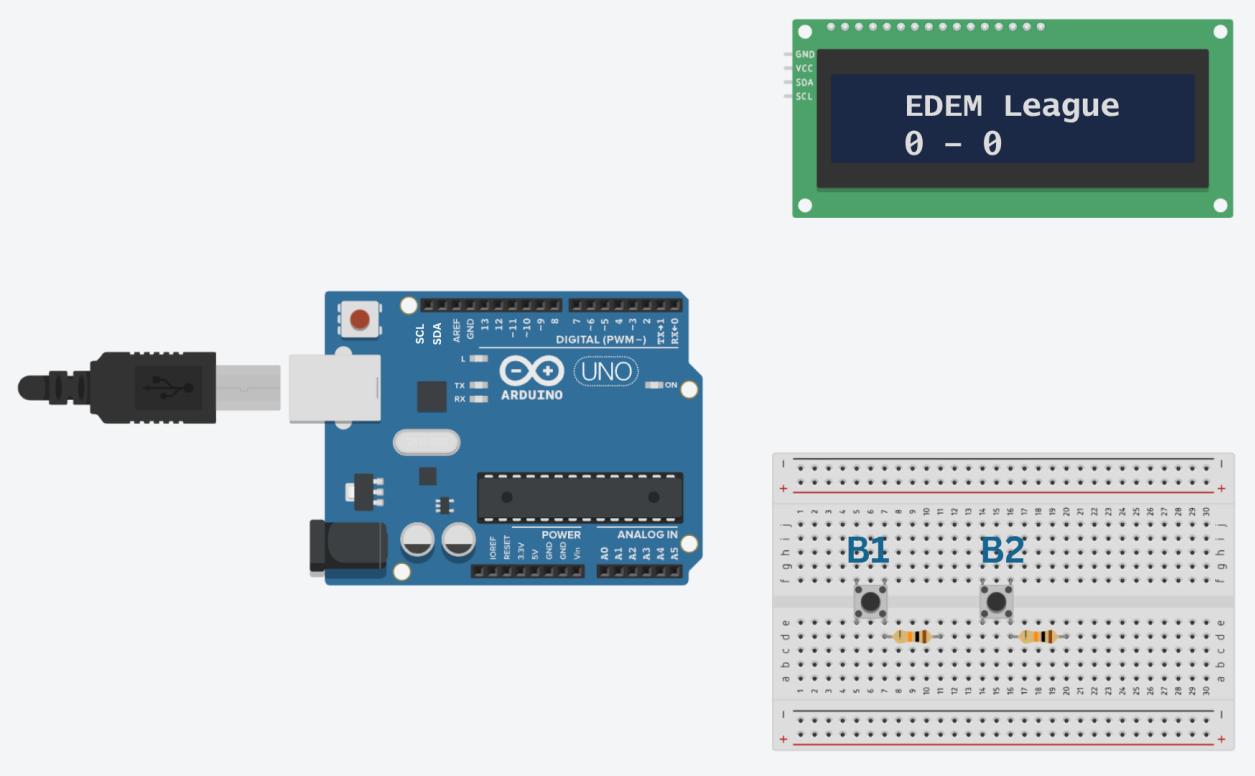



Figure 1: EDEM League electronic table

Based on the provided information and diagram, follow the instructions below to complete the wiring of the prototype.

1. Draw over the diagram the wires needed to connect the circuit (0.5 pts)
   (a) Complete the variables, constants needed. (0.5 pts)
   (b) Complete the setup function (1 pts)
   (c) Complete the loop function (1.5 pts)

To write the code, you can base it on this example of how an LCD screen works, provided as an example in the next page.

## Example code

This code will turn on the LCD screen, write ”hello world” in the first line, and then, in the second line, it writes, every second, the amount of seconds that it has been working. You can use this code as inspiration to write the code of the last exercise.

```
#include <Adafruit_LiquidCrystal.h>
int seconds = 0;
Adafruit_LiquidCrystal lcd_1(0);
void setup()
{
    lcd_1.begin(16, 2);
    lcd_1.print("hello world");
}
void loop()
{
    lcd_1.setCursor(0, 1);
    lcd_1.print(seconds);
    delay(1000); // Wait for 1000 milliseconds
    seconds += 1;
}
```